Jenkins Foundations

Architecture, Installation, UI Overview, Plugins, Security, Users, and Roles

Full Theoretical Notes - Stepwise, Part by Part

Purpose: This PDF explains Jenkins from first to last in a systematic manner. It is written for students who need clear theory before performing Jenkins practicals.

Learning Outcome: After completing this material, a student will understand Jenkins architecture, installation flow, web interface, plugin management, security configuration, user management, and role-based access control.

Part 1: Introduction to Jenkins

Jenkins is an open-source automation server used mainly for Continuous Integration and Continuous Delivery. It helps developers automatically build, test, package, and deploy applications. Instead of performing repeated tasks manually, Jenkins executes them through jobs or pipelines. This saves time, reduces human error, and improves software quality.

In modern DevOps, Jenkins works as a central automation tool. Developers push code to a repository such as GitHub. Jenkins detects the change, downloads the latest code, runs build commands, executes tests, and can deploy the application to a server or cloud platform.

1.1 Why Jenkins is Important

- It automates repetitive software development tasks.
- It supports Continuous Integration and Continuous Delivery.
- It reduces manual errors during build and deployment.
- It provides build history, logs, and execution status.
- It integrates with Git, Maven, Gradle, Docker, Kubernetes, and cloud platforms.
- It allows teams to deliver software faster and more reliably.

1.2 Basic Jenkins Workflow

1. Developer writes or modifies source code.
2. Developer pushes the code to a version control system such as GitHub.
3. Jenkins detects the code change manually, by webhook, or by scheduled polling.
4. Jenkins starts a job or pipeline.
5. Jenkins downloads the latest source code into the workspace.
6. Jenkins runs build commands such as Maven, Gradle, npm, or shell commands.
7. Jenkins runs automated tests.
8. Jenkins stores logs, reports, and build status.
9. If configured, Jenkins deploys the application to a target environment.

Part 2: Jenkins Architecture - Controller/Agent Model

Jenkins uses a distributed architecture called the Controller/Agent model. Older materials often use the term Master/Slave, but the modern Jenkins terminology is Controller/Agent. The controller manages Jenkins, while agents execute build workloads.

2.1 Main Components of Jenkins Architecture

Component	Meaning	Main Responsibility
Controller	Central Jenkins server	Manages UI, jobs, configuration, users, plugins, queue, and agents
Agent	Worker machine connected to controller	Executes build, test, and deployment tasks
Executor	Build slot on controller or agent	Runs one build at a time
Build Queue	Waiting area for jobs	Holds jobs until an executor is available
Workspace	Job working directory	Stores source code and temporary build files
Plugins	Extension modules	Add integrations and extra features
Credentials Store	Secure secrets area	Stores passwords, tokens, SSH keys, and certificates

2.2 Jenkins Controller

The Jenkins controller is the main server that controls the Jenkins environment. It provides the web interface, stores job configuration, manages plugins, handles security, schedules builds, and communicates with agents. The controller should mainly coordinate work, not run heavy builds.

Responsibilities of the Controller

- Provides Jenkins web dashboard and administration pages.
- Stores job configuration and build records.
- Controls authentication and authorization.
- Manages installed plugins and global tools.
- Places jobs in the build queue.
- Assigns jobs to available executors.
- Monitors agent connectivity and status.
- Stores logs, reports, artifacts, and job metadata.

2.3 Jenkins Agent

A Jenkins agent is a separate machine or container connected to the controller. Agents perform the actual build and test work. Using agents improves scalability because many jobs can run in parallel on different machines.

Types of Agents

- Permanent Agent: A fixed machine that remains connected to Jenkins.
- SSH Agent: A Linux or remote machine connected through SSH.
- Windows Agent: A Windows machine used for Windows-specific builds.
- Docker Agent: A temporary container used to run pipeline stages.
- Cloud Agent: A dynamically created machine or pod in cloud or Kubernetes.

2.4 How Controller and Agent Work Together

10. A user or trigger starts a Jenkins job.
11. The controller checks the job configuration.
12. The job enters the build queue.
13. The controller searches for a free executor.
14. If the job is restricted to a label, Jenkins selects a matching agent.
15. The controller sends the job to the selected agent.
16. The agent creates or reuses a workspace.
17. The agent runs build commands and test commands.

18. The agent sends logs and results back to the controller.
19. The controller displays the final result as success, failure, unstable, or aborted.

2.5 Advantages of Controller/Agent Architecture

- Scalability: Multiple agents can run multiple jobs at the same time.
- Performance: Heavy build work is moved away from the controller.
- Platform Support: Different agents can run Windows, Linux, or macOS builds.
- Isolation: Builds can run in separate systems or containers.
- Better Resource Use: Powerful machines can be assigned to demanding jobs.

Part 3: Jenkins Installation - Theoretical Understanding

Jenkins can be installed on Windows, Linux, macOS, Docker, or Kubernetes. For beginners, Windows installation is common because it provides an installer and service-based setup. The installation process requires Java because Jenkins is a Java-based application.

3.1 Installation Requirements

- Operating System: Windows, Linux, or macOS.
- Java: Modern Jenkins versions require a supported Java version, usually Java 17 or Java 21 depending on the Jenkins release.
- RAM: Minimum 4 GB recommended for learning environments.
- Storage: At least 20 GB recommended for Jenkins home, logs, jobs, and plugins.
- Browser: Chrome, Firefox, Edge, or any modern browser.
- Network: Required for downloading plugins and connecting to Git repositories.

3.2 Important Installation Concepts

Concept	Explanation
Jenkins Home	Main directory where Jenkins stores jobs, plugins, users, credentials, and configuration.
Port 8080	Default web port used to access Jenkins in a browser.
Initial Admin Password	Temporary password used to unlock Jenkins after first installation.
Suggested Plugins	Common plugins recommended for basic Jenkins usage.
Admin User	First permanent user created during setup.
Windows Service	Background service that starts Jenkins automatically on Windows.

3.3 Stepwise Installation Flow

20. Install a supported Java Development Kit on the system.
21. Verify Java installation from Command Prompt using java -version.
22. Download the Jenkins installer from the official Jenkins website.
23. Run the installer and select installation path.
24. Allow Jenkins to run as a Windows service if using Windows.
25. Open Jenkins in a browser using http://localhost:8080.
26. Copy the initial administrator password from the secrets folder.
27. Paste the password in the Jenkins unlock screen.
28. Install suggested plugins for basic Jenkins functionality.
29. Create the first admin user.
30. Confirm the Jenkins URL.
31. Open the Jenkins dashboard and begin configuration.

Part 4: Jenkins User Interface Overview

The Jenkins user interface is web-based. After logging in, the user reaches the Jenkins dashboard. From this dashboard, users create jobs, check build status, view logs, manage plugins, configure security, and administer Jenkins.

4.1 Main Dashboard Areas

UI Area	Purpose
New Item	Creates a new Jenkins job such as Freestyle, Pipeline, or Multibranch Pipeline.
People	Displays Jenkins users and user-related information.
Build History	Shows recent builds and their status.
Manage Jenkins	Central administration page for system, plugins, tools, security, and nodes.
Credentials	Stores and manages secrets for Git, servers, APIs, and deployments.
Build Queue	Shows jobs waiting for executors.
Executor Status	Shows which controller or agent executors are running jobs.

4.2 Jenkins Job Page

Each Jenkins job has its own page. This page contains job configuration, build history, workspace, console output, and options to build or delete the job. The job page is important because most Jenkins activities happen inside jobs or pipelines.

- Build Now: Starts the job manually.
- Configure: Changes job settings.
- Workspace: Shows files used during the job.
- Build History: Shows previous builds.
- Console Output: Displays real-time build logs.
- Changes: Shows source code changes if linked with version control.
- Delete Project: Removes the job from Jenkins.

4.3 Build Status Meaning

Status	Meaning
Success	Build completed without errors.
Failure	Build failed due to errors in build, test, or deployment steps.
Unstable	Build completed but tests or quality checks failed.
Aborted	Build was stopped manually or by system condition.
Not Built	Job or stage did not run.

Part 5: Plugins Management

Plugins are extension modules that add new features to Jenkins. Jenkins is powerful because of its plugin ecosystem. Without plugins, Jenkins provides only basic automation features. With plugins, Jenkins can connect to GitHub, Docker, Maven, Kubernetes, Slack, email systems, code scanners, and many other tools.

5.1 Why Plugins are Needed

- To integrate Jenkins with source code repositories.
- To support build tools such as Maven, Gradle, and npm.
- To create pipelines using Jenkinsfile.
- To connect Jenkins with Docker and Kubernetes.

- To send notifications by email or chat tools.
- To add security and role-based access features.
- To generate test reports, coverage reports, and dashboards.

5.2 Common Jenkins Plugins

Plugin	Use
Git Plugin	Allows Jenkins to clone and build projects from Git repositories.
Pipeline Plugin	Enables Jenkins Pipeline and Jenkinsfile-based automation.
Maven Integration Plugin	Supports Maven project builds.
Credentials Plugin	Manages usernames, passwords, tokens, and keys.
Role Strategy Plugin	Adds role-based authorization.
Docker Pipeline Plugin	Allows Docker commands inside pipelines.
Email Extension Plugin	Sends advanced email notifications.
JUnit Plugin	Publishes test reports.

5.3 Plugin Management Steps

32. Open Jenkins dashboard.
33. Click Manage Jenkins.
34. Click Plugins or Plugin Manager.
35. Use Available Plugins to install new plugins.
36. Use Installed Plugins to view existing plugins.
37. Use Updates to update outdated plugins.
38. Use Advanced Settings if proxy or update site configuration is required.
39. Restart Jenkins if a plugin requires restart.
40. After installation, configure plugin settings if required.

5.4 Plugin Safety Rules

- Install only required plugins.
- Prefer popular and actively maintained plugins.
- Keep plugins updated to avoid security vulnerabilities.
- Test plugin updates in a non-production Jenkins first.
- Remove unused plugins to reduce risk and complexity.
- Check plugin dependencies before uninstalling anything.

Part 6: Jenkins Security

Jenkins security is very important because Jenkins often has access to source code, credentials, build servers, deployment servers, containers, cloud accounts, and production systems. A poorly secured Jenkins server can become a major risk for an organization.

6.1 Core Security Concepts

Concept	Meaning
Authentication	Confirms who the user is.
Authorization	Controls what the user can do.
Security Realm	Defines where user identities come from.
Authorization Strategy	Defines permission rules.
Credentials Management	Stores secrets securely.
CSRF Protection	Protects Jenkins from unauthorized web requests.
Agent Security	Controls how agents connect and execute jobs.

6.2 Authentication in Jenkins

Authentication verifies user identity. Jenkins can authenticate users through its own user database or external systems such as LDAP, Active Directory, GitHub OAuth, or Single Sign-On.

- Jenkins own user database: Simple option for small labs and learning environments.
- LDAP or Active Directory: Used in organizations to manage users centrally.
- GitHub OAuth: Allows login using GitHub accounts.
- SSO: Used in enterprise environments for centralized login.

6.3 Authorization in Jenkins

Authorization controls permissions after login. It decides whether a user can create jobs, configure jobs, run builds, manage Jenkins, view credentials, or administer users.

- Logged-in users can do anything: Easy but unsafe for real environments.
- Matrix-based security: Permissions are assigned user by user or group by group.
- Project-based matrix security: Permissions are assigned per project.
- Role-based strategy: Permissions are grouped into roles and assigned to users.

6.4 Jenkins Security Best Practices

41. Disable anonymous access unless read-only public access is intentionally required.
42. Create individual user accounts instead of sharing admin credentials.
43. Give users only the permissions they need.
44. Use role-based access control for organized permission management.
45. Keep Jenkins core and plugins updated.
46. Use HTTPS for secure browser communication.
47. Store secrets only in Jenkins credentials store, not inside scripts.
48. Use separate agents for untrusted jobs.
49. Limit who can configure pipelines and execute scripts.
50. Take regular backups of Jenkins home directory.

Part 7: Users and Roles

User and role management allows Jenkins administrators to control access. In a student lab, one admin user may be enough. In a team or company, different users need different permissions. For example, an admin manages Jenkins, developers run jobs, testers view reports, and viewers only check status.

7.1 User Management

51. Open Jenkins dashboard.
52. Go to Manage Jenkins.
53. Open Security or Users section depending on Jenkins version.
54. Create a new user by entering username, password, full name, and email.
55. Assign permissions through the selected authorization strategy.
56. Ask the user to log in and verify access.
57. Review user permissions regularly.

7.2 Common Jenkins Roles

Role	Typical Permissions
Administrator	Full control of Jenkins, plugins, users, jobs, credentials, and nodes.
DevOps Engineer	Create and manage pipelines, agents, credentials, and deployments.

Developer	View jobs, run builds, read logs, and sometimes configure own projects.
Tester	View builds, check test reports, and run test jobs.
Viewer	Read-only access to jobs, logs, and dashboards.

7.3 Role Strategy Plugin Theory

The Role Strategy Plugin allows administrators to create roles and assign permissions to users or groups. It is useful because permissions are managed in a structured way. Instead of assigning permissions separately to every user, an administrator creates roles such as admin, developer, tester, and viewer, then assigns users to those roles.

7.4 Types of Roles in Jenkins

- Global Roles: Apply across the whole Jenkins system.
- Project Roles: Apply only to jobs that match a name pattern.
- Agent Roles: Apply to node or agent management permissions.

7.5 Example Role Design

Role Name	Access Level	Example Use
admin	Full Jenkins access	Faculty, lab administrator, DevOps lead
developer	Build and configure project jobs	Students or developers working on assigned projects
tester	Build and view test reports	Testing team members
viewer	Read-only access	Managers, reviewers, external observers

Part 8: Credentials Management

Credentials are sensitive values used by Jenkins to access external systems. Examples include GitHub tokens, Docker Hub passwords, SSH keys, server passwords, API keys, and certificates. Jenkins provides a credentials store to keep these secrets protected.

8.1 Types of Jenkins Credentials

Credential Type	Example Use
Username with password	Git, Docker Hub, server login
Secret text	API token or webhook secret
SSH username with private key	Remote Linux server access
Secret file	Certificate or configuration file
Certificate	Secure system authentication

8.2 Credentials Best Practices

- Do not write passwords directly in pipeline scripts.
- Use credentials binding to inject secrets only when required.
- Limit credential visibility using folders and scopes.
- Rotate tokens and passwords regularly.
- Remove unused credentials.
- Never print secrets in console output.

Part 9: Jenkins Jobs and Pipelines Overview

A Jenkins job is a task configured in Jenkins. A pipeline is a more advanced job that defines automation as code. Pipeline definitions are usually stored in a Jenkinsfile so that the build process can be version-controlled with the source code.

9.1 Types of Jenkins Jobs

Job Type	Meaning
Freestyle Project	Simple job configured mostly through the UI. Good for beginners.
Pipeline	Job written using Jenkins Pipeline syntax. Good for CI/CD workflows.
Multibranch Pipeline	Automatically creates pipelines for repository branches.
Folder	Organizes related jobs together.

9.2 Pipeline Stages

- Checkout: Download source code from repository.
- Build: Compile or package the application.
- Test: Run unit tests, integration tests, or quality checks.
- Package: Create deployable artifacts.
- Deploy: Move application to server, container, or cloud.
- Post Actions: Send notifications and archive reports.

Part 10: Complete Stepwise Learning Order for Students

Students should learn Jenkins in a proper sequence. The following order helps avoid confusion and builds understanding from basic theory to administration.

58. Understand what Jenkins is and why automation is needed.
59. Learn the difference between Continuous Integration and Continuous Delivery.
60. Understand Jenkins controller, agent, executor, queue, and workspace.
61. Install Java and Jenkins on the system.
62. Unlock Jenkins using the initial admin password.
63. Install suggested plugins.
64. Create the first admin user.
65. Explore the dashboard and Manage Jenkins page.
66. Create a simple Freestyle job.
67. View console output and build history.
68. Install and update plugins.
69. Configure basic security settings.
70. Create users for different roles.
71. Install Role Strategy Plugin for role-based access.
72. Create admin, developer, tester, and viewer roles.
73. Assign users to roles.
74. Create a basic Pipeline job.
75. Connect Jenkins with Git repository.
76. Run build and test stages.
77. Follow Jenkins security and backup best practices.

Part 11: Summary

Jenkins is a powerful automation server used in DevOps for building, testing, and deploying software. Its controller/agent architecture allows distributed builds and improves scalability. The Jenkins UI helps users create jobs, view logs, manage plugins, configure security, and control users. Plugins extend Jenkins functionality and connect it with modern tools. Security is essential because Jenkins often stores credentials and controls deployment systems. Proper user and role management ensures that every user gets only the access required for their work.

Part 12: Quick Revision Points

- Jenkins automates build, test, and deployment tasks.
- The controller manages Jenkins; agents run jobs.
- Executors are build slots that run jobs.
- Plugins add integrations and features.
- Jenkins dashboard is the main web interface.
- Security includes authentication, authorization, and credentials protection.
- Role Strategy Plugin supports role-based access control.
- Credentials should never be hardcoded in scripts.
- Pipelines define CI/CD as code using Jenkinsfile.
- Regular updates and backups are important for safe Jenkins operation.